

The Art of Writing Meaningful Git Commit Messages

 medium.com/@iambonitheuri/the-art-of-writing-meaningful-git-commit-messages-a56887a4cb49

theurikarue

August 9, 2024

Press enter or click to view image in full size

As you already know, I spent the last three months of my life in the JKUAT internal attachment program, and the articles I've written so far have shared the experiences I've had and the lessons I've learnt while there.

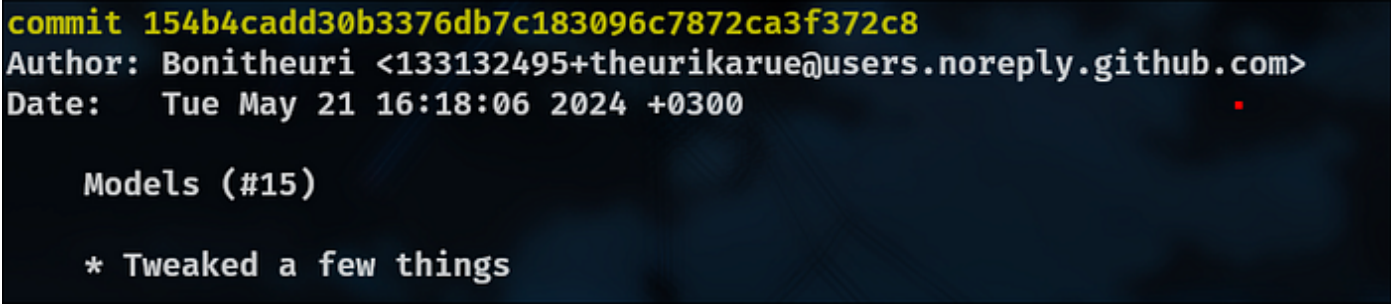
Today, I would like to talk about writing meaningful git commit messages, don't worry, it'll not be a long article. I tend to think there's nothing as hard as coming up with names for variables when it comes to programming and I believe the things that comes second is coming up with meaningful git commit messages -I would often hear

[Hilda Mwangi](#)

ask Florence (and vice versa) to help her come up with commit messages, and they'd proceed to laugh- as it's sometimes a daunting task.

On a personal note looking back at my old commits, I found a message from May 21st, 2024.

Press enter or click to view image in full size



```
commit 154b4cadd30b3376db7c183096c7872ca3f372c8
Author: Bonitheuri <133132495+theurikarue@users.noreply.github.com>
Date: Tue May 21 16:18:06 2024 +0300

    Models (#15)

    * Tweaked a few things
```

It simply said 'Tweaked a few things' — not exactly helpful! I mean, who says that in this time and age?

Why Should You Write Better Commit Messages.

I think Git enters a whole other realm the moment you start working in teams. I challenge you to open up a personal project, or any repository for that matter, and run `git log` to view a list of old commit messages. (Btw, that's how I've gotten to find my commit message from May 21st, 2024.) You'll get to realize that six months later, you definitely cannot remember what the few things you tweaked were. Now imagine if you were working in a team; do you think they would know what you meant by 'Tweaked a few things'? Probably not.

So, in a nutshell, you should write better commit messages for the following reasons:

- Future readers will not have trouble understanding what changed and why it changed.
- To make it easier to undo certain modifications.
- Making changes to the release notes.

Clear commit messages serve as a roadmap for future developers, making it easier to track changes and understand the project's evolution.

The Anatomy of A Good Commit Message.

Basic:

```
git commit -m <message>
```

Detailed:

```
git commit -m <title> <description>
```

In case you make an error in a recent commit you can run the following command

(allows you to modify and add changes to the most recent commit)

```
git commit --amend
```

Note: It's great to use amend to clean up local commits, and once you're done, you can push the updated version to a shared repository. However, it's best to refrain from changing commits that have already been made public.

Conventional Commits

Now that we've covered basic commit structure of a good commit message, I'd like to introduce conventional commits to help provide some detail on creating solid commit messages.

Join Medium for free to get updates from this writer.

Remember me for faster sign in

Here's a great template for a good commit message:.

```
<type>[optional scope]: <description> [optional body] [optional footer(s)]
```

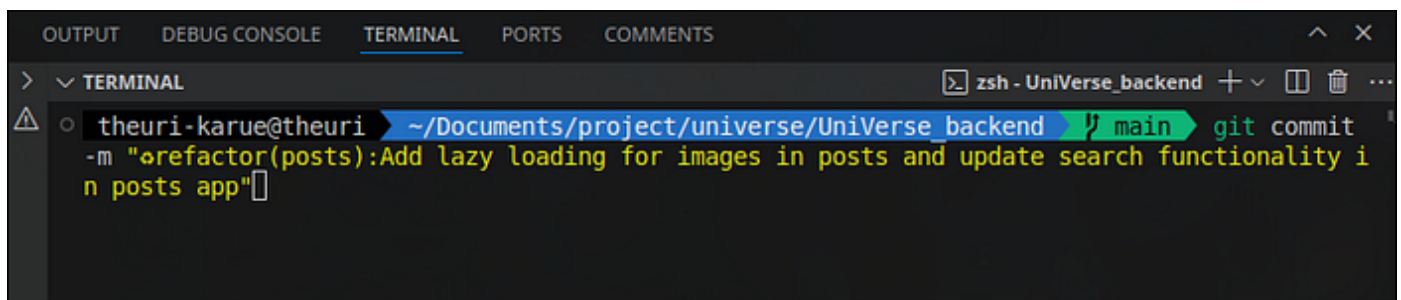
The commit type can be

- **feat**: Commits, which adds a new feature
- **fix**: Commits, that fixes a bug
- **refactor**: refactored code that neither fixes a bug nor adds a feature but rewrites/restructures your code.

- **chore**: Changes that do not relate to a fix or feature and don't modify src or test files basically miscellaneous commits (for example, updating dependencies or modifying .gitignore file)
- **perf**: Commits are special refactor commits, geared towards improving performance.
- **ci**: Continuous integration related.
- **ops**: Commits, that affect operational components like infrastructure, deployment, backup , recovery ...
- **build**: Changes that affect the build system build tool, ci pipeline, dependencies, project version, ...
- **docs**: Commits, that affect documentation, such as the README.
- **style**: changes that do not affect the meaning of the code, likely related to code formatting such as white-space, missing semi-colons, etc.
- **revert**: reverts a previous commit.
- **test**: commits that add missing tests or correct existing tests

eg:

Press enter or click to view image in full size



```

theuri-karue@theuri ~/Documents/project/universe/UniVerse_backend main git commit
-m "refactor(posts): Add lazy loading for images in posts and update search functionality i
n posts app"

```

Rules for creating a great commit message

- Limit the subject line to 50 characters
- Capitalize the subject/description line
- Do not end the subject line with a period
- Separate the subject from the body with a blank line
- Wrap the body at 72 characters
- Use the body to explain what and why
- Use the imperative mood in the subject line let it seem like you're giving a command eg "feat: Add unit tests for user authentication". Using the imperative mood in commit messages makes them more consistent and commands-like, which is helpful in understanding the actions taken.

Conclusion

Writing good commit messages is an extremely beneficial skill to develop, and it helps you communicate and collaborate with your team. Commits serve as an archive of changes. They can become an ancient manuscript to help us decipher the past, and make reasoned decisions in the future.

I hope this article has helped you go from 'Tweaked a few things' to meaningful messages that benefit everyone on your team.

References:

- [1] <https://www.freecodecamp.org/news/writing-good-commit-messages-a-practical-guide/>
- [2] <https://medium.com/front-end-weekly/how-to-write-good-git-commit-messages-like-a-pro-2c12f01569d9>
- [3] <https://kelvinromero.medium.com/writing-good-commit-messages-527679b1babb>